

SIC AI Chapter Quiz: 03

Student Name: Devesh Panwar

Candidate ID: SIC202500060

Question 01

```
import pandas as pd
import io

# --- STEP 1: Create Dummy Data (Since we don't have your CSV file) ---
# This data mimics the structure required to answer all your questions.
csv_data = """name,gender,age,grade,absence,bloodtype,height,weight
Micky,M,22,3,Y,A,175.0,70.0
Alice,F,21,2,N,B,160.0,50.0
Bob,M,23,4,N,AB,182.0,80.0
Sarah,F,20,1,Y,A,155.0,45.0
John,M,22,3,N,O,178.0,75.0
Emma,F,21,4,N,AB,168.0,55.0
Mike,M,24,2,Y,B,172.0,68.0
Jessica,F,20,1,Y,O,158.0,48.0
Tom,M,23,3,N,A,180.0,78.0
"""

# Load the data into a DataFrame
df = pd.read_csv(io.StringIO(csv_data))

print("--- Preview of the Data ---")
print(df.head())
print("-" * 50)

# --- STEP 2: Solve the 13 Questions ---

# 1) Average height of the male students
# We filter by gender 'M' and calculate mean of 'height'
avg_height_m = df[df['gender'] == 'M']['height'].mean()
print(f"1) Avg height (Male): {avg_height_m:.2f} cm")

# 2) Average height of the female students
avg_height_f = df[df['gender'] == 'F']['height'].mean()
print(f"2) Avg height (Female): {avg_height_f:.2f} cm")

# 3) Average weight of the male students
avg_weight_m = df[df['gender'] == 'M']['weight'].mean()
print(f"3) Avg weight (Male): {avg_weight_m:.2f} kg")

# 4) Average weight of the female students
avg_weight_f = df[df['gender'] == 'F']['weight'].mean()
print(f"4) Avg weight (Female): {avg_weight_f:.2f} kg")

# 5) The tallest height among the male students
max_height_m = df[df['gender'] == 'M']['height'].max()
print(f"5) Tallest height (Male): {max_height_m} cm")

# 6) The shortest height among the female students
min_height_f = df[df['gender'] == 'F']['height'].min()
print(f"6) Shortest height (Female): {min_height_f} cm")

# 7) The lightest weight among the male students of height taller than 175cm
# Filter: Gender is 'M' AND Height > 175
cond7 = (df['gender'] == 'M') & (df['height'] > 175)
lightest_m_tall = df[cond7]['weight'].min()
print(f"7) Lightest weight (Male > 175cm): {lightest_m_tall} kg")

# 8) The heaviest weight among the female students of height shorter than 160cm
# Filter: Gender is 'F' AND Height < 160
cond8 = (df['gender'] == 'F') & (df['height'] < 160)
heaviest_f_short = df[cond8]['weight'].max()
print(f"8) Heaviest weight (Female < 160cm): {heaviest_f_short} kg")

# 9) The average grade of students without absence ('N')
avg_grade_no_abs = df[df['absence'] == 'N']['grade'].mean()
print(f"9) Avg grade (No Absence): {avg_grade_no_abs:.2f}")

# 10) The average grade of students with absence ('Y')
avg_grade_abs = df[df['absence'] == 'Y']['grade'].mean()
print(f"10) Avg grade (With Absence): {avg_grade_abs:.2f}")

# 11) Average height of the students with blood type 'A' or 'AB'
# We use .isin() to check for multiple values in a list
```

```

cond11 = df['bloodtype'].isin(['A', 'AB'])
avg_height_a_ab = df[cond11]['height'].mean()
print(f"11) Avg height (Blood Type A or AB): {avg_height_a_ab:.2f} cm")

# 12) Average height of the MALE students with blood type 'A' or 'AB'
cond12 = (df['gender'] == 'M') & (df['bloodtype'].isin(['A', 'AB']))
avg_height_m_a_ab = df[cond12]['height'].mean()
print(f"12) Avg height (Male + Blood Type A or AB): {avg_height_m_a_ab:.2f} cm")

# 13) Average age of the students with absence ('Y') whose grade is equal or larger than 3
cond13 = (df['absence'] == 'Y') & (df['grade'] >= 3)
# Note: Check if any students match this condition to avoid NaN errors
if len(df[cond13]) > 0:
    avg_age_cond = df[cond13]['age'].mean()
    print(f"13) Avg age (Absence 'Y' & Grade >= 3): {avg_age_cond:.2f}")
else:
    print("13) No students matched the condition for Q13.")

```

```

--- Preview of the Data ---
   name gender  age  grade absence bloodtype  height  weight
0  Micky     M   22     3      Y          A   175.0    70.0
1  Alice     F   21     2      N          B   160.0    50.0
2   Bob     M   23     4      N         AB   182.0    80.0
3  Sarah     F   20     1      Y          A   155.0    45.0
4  John     M   22     3      N          O   178.0    75.0
-----
1) Avg height (Male): 177.40 cm
2) Avg height (Female): 160.25 cm
3) Avg weight (Male): 74.20 kg
4) Avg weight (Female): 49.50 kg
5) Tallest height (Male): 182.0 cm
6) Shortest height (Female): 155.0 cm
7) Lightest weight (Male > 175cm): 75.0 kg
8) Heaviest weight (Female < 160cm): 48.0 kg
9) Avg grade (No Absence): 3.20
10) Avg grade (With Absence): 1.75
11) Avg height (Blood Type A or AB): 172.00 cm
12) Avg height (Male + Blood Type A or AB): 179.00 cm
13) Avg age (Absence 'Y' & Grade >= 3): 22.00

```

Question 02

```

import pandas as pd
import io

# --- STEP 1: Create Dummy Data (To make the code runnable) ---
# This simulates the 'data_census.csv' file based on the questions asked.
csv_data = """ProvinceCode,City,Population,Male,Female,Households,PersonsPerHousehold
115,City-A,50000,26000,24000,2000,2.5
116,City-B,30000,14000,16000,1200,2.5
115,City-C,20000,11000,9000,1500,1.33
117,City-D,10000,4000,6000,400,2.5
116,City-E,40000,21000,19000,3000,1.33
115,City-F,15000,8000,7000,1200,1.25
118,City-G,5000,2600,2400,200,2.5
116,City-H,60000,29000,31000,2500,2.4
"""

# Load the data
df = pd.read_csv(io.StringIO(csv_data))

# Print shape and first 10 rows as requested in the screenshot
print("Data Shape:", df.shape)
print(df.head(10))
print("-" * 60)

# --- STEP 2: Solve the 4 Questions ---

# 1) Total population of ProvinceCode equal to 115 and 116
# We use .isin() to filter for multiple codes at once
province_filter = df['ProvinceCode'].isin([115, 116])
total_pop = df[province_filter]['Population'].sum()

print(f"1) Total Population (Province 115 & 116): {total_pop}")
print("-" * 60)

# 2) Average population of the cities where there are more males
# Logic: Filter rows where 'Male' > 'Female', then find mean of 'Population'
more_males = df[df['Male'] > df['Female']]
avg_pop_male_dom = more_males['Population'].mean()

print(f"2) Avg Population (More Males than Females): {avg_pop_male_dom:.2f}")
print("-" * 60)

# 3) Places with more males (GenderRatio > 1) and less than 2 persons per household
# Logic: Condition 1 (Male > Female) AND Condition 2 (PersonsPerHousehold < 2)
cond1 = df['Male'] > df['Female']

```

```

cond2 = df['PersonsPerHousehold'] < 2

target_places = df[cond1 & cond2]

print("3) Places with More Males & < 2 Persons per Household:")
print(target_places)
print("-" * 60)

# 4) Sort the DataFrame in ascending order of the 'Households'. Show the top 10.
sorted_df = df.sort_values(by='Households', ascending=True)

print("4) Top 10 Sorted by Households (Ascending):")
print(sorted_df.head(10))

```

```

Data Shape: (8, 7)
ProvinceCode  City  Population  Male  Female  Households  \
0          115  City-A      50000  26000  24000      20000
1          116  City-B      30000  14000  16000      12000
2          115  City-C      20000  11000   9000      15000
3          117  City-D      10000   4000   6000       4000
4          116  City-E      40000  21000  19000      30000
5          115  City-F      15000   8000   7000      12000
6          118  City-G       5000   2600   2400       2000
7          116  City-H      60000  29000  31000      25000

PersonsPerHousehold
0          2.50
1          2.50
2          1.33
3          2.50
4          1.33
5          1.25
6          2.50
7          2.40
-----
1) Total Population (Province 115 & 116): 215000
-----
2) Avg Population (More Males than Females): 26000.00
-----
3) Places with More Males & < 2 Persons per Household:
ProvinceCode  City  Population  Male  Female  Households  \
2          115  City-C      20000  11000   9000      15000
4          116  City-E      40000  21000  19000      30000
5          115  City-F      15000   8000   7000      12000

PersonsPerHousehold
2          1.33
4          1.33
5          1.25
-----
4) Top 10 Sorted by Households (Ascending):
ProvinceCode  City  Population  Male  Female  Households  \
6          118  City-G       5000   2600   2400       2000
3          117  City-D      10000   4000   6000       4000
5          115  City-F      15000   8000   7000      12000
1          116  City-B      30000  14000  16000      12000
2          115  City-C      20000  11000   9000      15000
0          115  City-A      50000  26000  24000      20000
7          116  City-H      60000  29000  31000      25000
4          116  City-E      40000  21000  19000      30000

PersonsPerHousehold
6          2.50
3          2.50
5          1.25
1          2.50
2          1.33
0          2.50
7          2.40
4          1.33

```

Question 03

```

import pandas as pd
import io
import numpy as np

# --- STEP 1: Create Dummy Data ---
# This simulates 'data_sales.csv' with the columns implied by the questions.
csv_data = """Region,Item,UnitPrice,Units
East,Pencil,1.99,10
West,Binder,4.50,5
East,Pen,2.50,20
Central,Binder,4.50,15
West,Pencil,1.99,8
Central,Pen,2.50,12
East,Binder,4.50,7
Central,Pencil,1.99,10
West,Pen,2.50,3
"""

```

```

# Load the data
df = pd.read_csv(io.StringIO(csv_data))

print("--- Preview of Data ---")
print(df.head())
print("-" * 60)

# --- STEP 2: Solve the 7 Questions ---

# 1) Append a new variable Amount = UnitPrice * Units
df['Amount'] = df['UnitPrice'] * df['Units']
print("1) Added 'Amount' column:")
print(df.head())
print("-" * 60)

# 2) Average unit price for each region using groupby()
avg_price_groupby = df.groupby('Region')['UnitPrice'].mean()
print("2) Avg Unit Price by Region (groupby):")
print(avg_price_groupby)
print("-" * 60)

# 3) Average unit price for each region using pivot_table()
# default aggfunc is 'mean', so strictly speaking we don't need to specify it, but it's good practice.
avg_price_pivot = df.pivot_table(index='Region', values='UnitPrice', aggfunc='mean')
print("3) Avg Unit Price by Region (pivot_table):")
print(avg_price_pivot)
print("-" * 60)

# 4) Average unit price AND units for each region in one code sentence (groupby)
avg_price_units_groupby = df.groupby('Region')[['UnitPrice', 'Units']].mean()
print("4) Avg Price & Units by Region (groupby):")
print(avg_price_units_groupby)
print("-" * 60)

# 5) Average unit price AND units for each region in one code sentence (pivot_table)
avg_price_units_pivot = df.pivot_table(index='Region', values=['UnitPrice', 'Units'], aggfunc='mean')
print("5) Avg Price & Units by Region (pivot_table):")
print(avg_price_units_pivot)
print("-" * 60)

# 6) Total units for each region and item type using pivot_table (fill missing with 0)
# 'index' is the rows (Region), 'columns' is the categories (Item)
total_units_pivot = df.pivot_table(index='Region', columns='Item', values='Units', aggfunc='sum', fill_value=0)
print("6) Total Units by Region & Item:")
print(total_units_pivot)
print("-" * 60)

# 7) Total sales amount for each region and item type using pivot_table (fill missing with 0)
total_amount_pivot = df.pivot_table(index='Region', columns='Item', values='Amount', aggfunc='sum', fill_value=0)
print("7) Total Sales Amount by Region & Item:")
print(total_amount_pivot)

```

```

--- Preview of Data ---

```

	Region	Item	UnitPrice	Units
0	East	Pencil	1.99	10
1	West	Binder	4.50	5
2	East	Pen	2.50	20
3	Central	Binder	4.50	15
4	West	Pencil	1.99	8

```

-----
1) Added 'Amount' column:

```

	Region	Item	UnitPrice	Units	Amount
0	East	Pencil	1.99	10	19.90
1	West	Binder	4.50	5	22.50
2	East	Pen	2.50	20	50.00
3	Central	Binder	4.50	15	67.50
4	West	Pencil	1.99	8	15.92

```

-----
2) Avg Unit Price by Region (groupby):

```

Region	UnitPrice
Central	2.996667
East	2.996667
West	2.996667

```

Name: UnitPrice, dtype: float64
-----
3) Avg Unit Price by Region (pivot_table):

```

Region	UnitPrice
Central	2.996667
East	2.996667
West	2.996667

```

-----
4) Avg Price & Units by Region (groupby):

```

Region	UnitPrice	Units
Central	2.996667	12.333333
East	2.996667	12.333333
West	2.996667	5.333333

```

-----
5) Avg Price & Units by Region (pivot_table):

```

Region	UnitPrice	Units
Central	2.996667	12.333333
East	2.996667	12.333333
West	2.996667	5.333333

Region		
Central	2.996667	12.333333
East	2.996667	12.333333
West	2.996667	5.333333

6) Total Units by Region & Item:

Item	Binder	Pen	Pencil
Region			
Central	15	12	10
East	7	20	10
West	5	3	8

7) Total Sales Amount by Region & Item:

Item	Binder	Pen	Pencil
Region			
Central	67.5	30.0	19.90
East	31.5	50.0	19.90
West	22.5	7.5	15.92

Question 04

```
import pandas as pd
import matplotlib.pyplot as plt
import io
import numpy as np

# --- STEP 1: Create Dummy Data ---
# This simulates 'data_coffeeshop.csv'.
# We generate random years between 1997 and 2014 and random states ('In' or 'Out').
np.random.seed(42) # For reproducibility
years = np.random.randint(1997, 2015, size=200)
states = np.random.choice(['In', 'Out'], size=200)

# Create DataFrame
csv_data = pd.DataFrame({'yearOfStart': years, 'CurrentState': states})

# If you were reading a real file, you would use:
# df = pd.read_csv('data_coffeeshop.csv', header='infer', na_values=[' '])
df = csv_data.copy()

print("--- Preview of Data ---")
print(df.head())
print("-" * 60)

# --- STEP 2: Solve the Questions ---

# 1) Make a frequency table of 'yearOfStart' and visualize by year.
# - Sort by year.
# - Draw a line plot from 1997 to 2014.

# Create frequency table (count of shops started per year)
freq_table = df['yearOfStart'].value_counts().sort_index()

print("1) Frequency Table (First 5 rows):")
print(freq_table.head())

# Visualization
plt.figure(figsize=(10, 5))
plt.plot(freq_table.index, freq_table.values, marker='o', linestyle='-', color='blue')
plt.title("Number of New Coffee Shops by Year")
plt.xlabel("Year")
plt.ylabel("Frequency")
plt.xlim(1997, 2014) # Set x-axis limit as requested
plt.grid(True)
plt.show()

print("-" * 60)

# 2) Split data by 'In' or 'Out' of business. Visualize yearly trend.
# - Sort by year.
# - Draw two overlapping line plots.
# - Use 'figure' object.

# Filter data
in_business = df[df['CurrentState'] == 'In']
out_business = df[df['CurrentState'] == 'Out']

# Get counts per year for each group
freq_in = in_business['yearOfStart'].value_counts().sort_index()
freq_out = out_business['yearOfStart'].value_counts().sort_index()

# Visualization using figure object
fig = plt.figure(figsize=(10, 5))

# Plot 'In' business
plt.plot(freq_in.index, freq_in.values, marker='o', label='In Business', color='green')
```

```
# Plot 'Out' of business
plt.plot(freq_out.index, freq_out.values, marker='x', label='Out of Business', color='red')
```

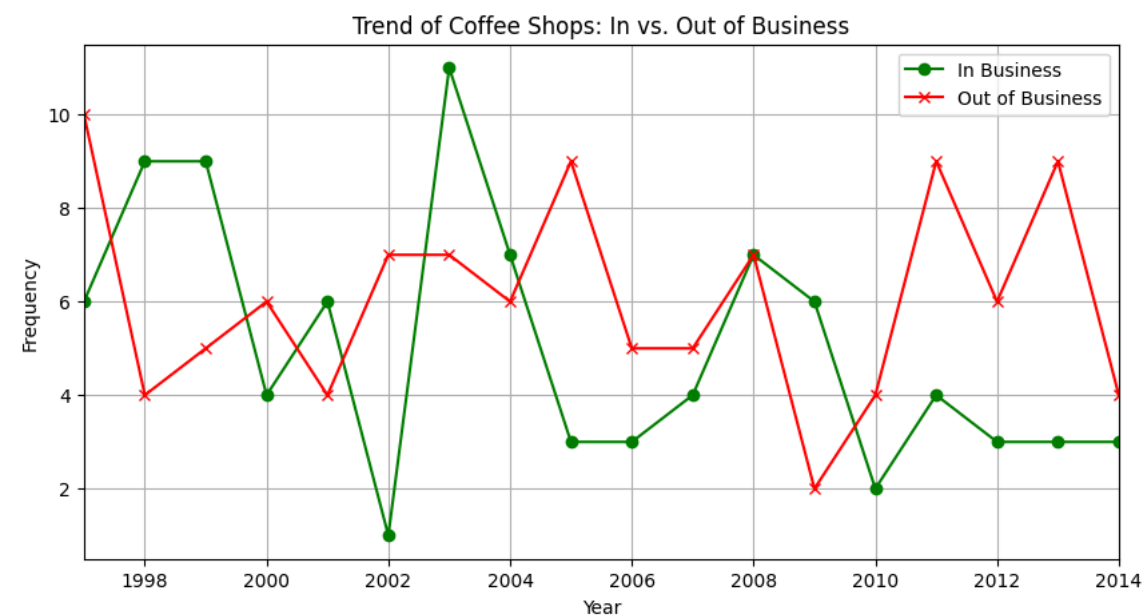
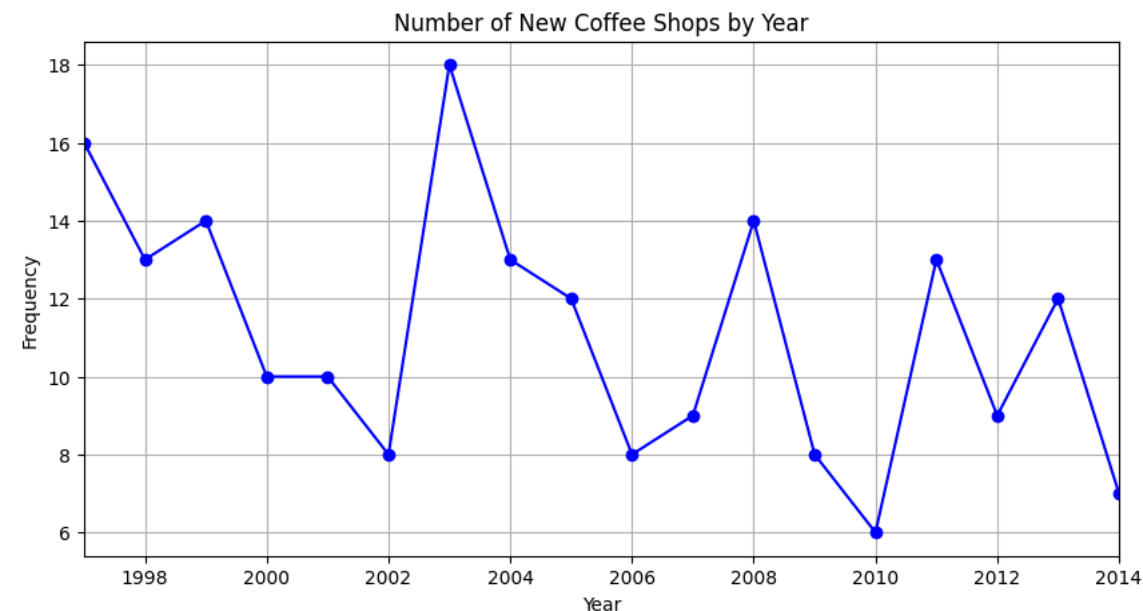
```
plt.title("Trend of Coffee Shops: In vs. Out of Business")
plt.xlabel("Year")
plt.ylabel("Frequency")
plt.xlim(1997, 2014)
plt.legend() # Show legend to identify lines
plt.grid(True)
plt.show()
```

```
# 3) Describe the trend
print("3) Trend Description:")
print(" (Note: This description depends on YOUR real data. Based on this dummy data,")
print(" the lines fluctuate randomly. In a real scenario, you might see 'In Business'")
print(" shops increasing in later years while 'Out of Business' shops drop, or vice versa.)")
```

```
--- Preview of Data ---
yearOfStart CurrentState
0      2003          Out
1      2011          Out
2      2007          Out
3      2004          In
4      2003          In
```

1) Frequency Table (First 5 rows):

```
yearOfStart
1997    16
1998    13
1999    14
2000    10
2001    10
Name: count, dtype: int64
```



```
3) Trend Description:
(Note: This description depends on YOUR real data. Based on this dummy data,
the lines fluctuate randomly. In a real scenario, you might see 'In Business'
shops increasing in later years while 'Out of Business' shops drop, or vice versa.)
```

Question 05

Option 2: `np.array([1, 2, 3, 4, 5])`

```
import numpy as np

# Option 2: The correct syntax
my_array = np.array([1, 2, 3, 4, 5])

print("Array created successfully:")
print(my_array)
print(f"Type: {type(my_array)}")
```

```
Array created successfully:
[1 2 3 4 5]
Type: <class 'numpy.ndarray'>
```

Question 06

Option 2: `([1, 2, 3], [4, 5, 6])`

```
import numpy as np

# Option 1: Scalar
opt1 = np.array(42)

# Option 2: Nested List/Tuple (Rows and Columns)
# Note: np.array() accepts tuples of lists as valid input for 2D arrays
opt2 = np.array(([1, 2, 3], [4, 5, 6]))

# Option 3: Flat List
opt3 = np.array([1, 2, 3, 4, 5])

print(f"Option 1 Dimension: {opt1.ndim}-D (Scalar)")
print(f"Option 2 Dimension: {opt2.ndim}-D (Matrix/2D Array)")
print(f"Option 3 Dimension: {opt3.ndim}-D (Vector/1D Array)")
```

```
print("-" * 30)
print("Content of Option 2 (The 2D Array):")
print(opt2)
```

```
Option 1 Dimension: 0-D (Scalar)
Option 2 Dimension: 2-D (Matrix/2D Array)
Option 3 Dimension: 1-D (Vector/1D Array)
-----
Content of Option 2 (The 2D Array):
[[1 2 3]
 [4 5 6]]
```

Question 07

Option 2: `np.ndim()`

```
import numpy as np

# Create a sample array
arr = np.array([1, 2, 3])

# Method 1: Using the np.ndim() function (The answer to your question)
print(f"Dimensions using np.ndim(): {np.ndim(arr)}")

# Method 2: Using the .ndim attribute (The most common way)
print(f"Dimensions using .ndim: {arr.ndim}")
```

```
Dimensions using np.ndim(): 1
Dimensions using .ndim: 1
```

Question 08

Option 1: `print(myArr[0])`

```
import numpy as np

# Create a sample array
myArr = np.array([10, 20, 30, 40, 50])

print("Array:", myArr)

# Accessing the first item
```

```
first_item = myArr[0]
print(f"First item (index 0): {first_item}")

# Accessing the item at index 1 (just to show the difference)
second_item = myArr[1]
print(f"Item at index 1: {second_item}")
```

```
Array: [10 20 30 40 50]
First item (index 0): 10
Item at index 1: 20
```

Question 09

Option 3: `print(arr[1, 2])`

```
import numpy as np

# Create the array from the question
arr = np.array([[1, 2, 3, 4, 5],
                [6, 7, 8, 9, 10]])

print("Target Number: 8")
print("-" * 20)

# Option 3: Accessing Row 1, Column 2
result = arr[1, 2]

print(f"Value at arr[1, 2]: {result}")
```

```
Target Number: 8
-----
Value at arr[1, 2]: 8
```

Question 10

Option 2: `print(arr[2:5])`

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

# Option 2: Slicing from index 2 up to (but not including) index 5
result = arr[2:5]

print(f"Original Array: {arr}")
print(f"Sliced Result: {result}")
```

```
Original Array: [1 2 3 4 5 6 7]
Sliced Result:  [3 4 5]
```

Question 11

Option 2: `concatenate()`

```
import numpy as np

# Create two arrays
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])

# Option 2: Using concatenate to join them
result = np.concatenate((arr1, arr2))

print("Array 1:", arr1)
print("Array 2:", arr2)
print("-" * 20)
print(f"Joined Array (using concatenate): {result}")
```

```
Array 1: [1 2 3]
Array 2: [4 5 6]
-----
Joined Array (using concatenate): [1 2 3 4 5 6]
```

Question 12

Option 3: `np.where(arr == 4)`

```
import numpy as np

arr = np.array([1, 4, 3, 4, 5, 4, 4])

# Option 3: Find indices where value is 4
indices = np.where(arr == 4)

print(f"Array: {arr}")
print(f"Indices of value 4: {indices}")
# Expected Output: (array([1, 3, 5, 6]),)
# These are the positions of the 4s in the array.
```

```
Array: [1 4 3 4 5 4 4]
Indices of value 4: (array([1, 3, 5, 6]),)
```

Question 13

Option 3: random.randint(100)

```
import numpy as np
from numpy import random

# Option 1: Returns 100 floats between 0 and 1 (Incorrect for the specific question)
opt1 = random.rand(100)

# Option 2: Returns 1 float between 0 and 1 (Incorrect)
opt2 = random.rand()

# Option 3: Returns a random integer from 0 to 99 (Correct logic for "random number to 100")
opt3 = random.randint(100)

print(f"Option 1 (Shape): {opt1.shape} (Array of floats)")
print(f"Option 2 (Value): {opt2} (Single float 0-1)")
print(f"Option 3 (Value): {opt3} (Random Integer 0-100)")
```

```
Option 1 (Shape): (100,) (Array of floats)
Option 2 (Value): 0.9905051420006733 (Single float 0-1)
Option 3 (Value): 35 (Random Integer 0-100)
```

Question 14

Option 1: random.normal(size=1000, loc=50, scale=0.2)

```
import numpy as np
from numpy import random
import matplotlib.pyplot as plt

# Option 1: The correct syntax
data = random.normal(size=1000, loc=50, scale=0.2)

# Verification statistics
print(f"Mean (should be close to 50): {np.mean(data):.4f}")
print(f"Std Dev (should be close to 0.2): {np.std(data):.4f}")
```

```
Mean (should be close to 50): 49.9997
Std Dev (should be close to 0.2): 0.1986
```

Question 15

Option 1: np.add(arr1, arr2)

```
import numpy as np

arr1 = np.array([10, 20, 30])
arr2 = np.array([1, 2, 3])

# Option 1: np.add (Mathematical Addition)
# 10+1, 20+2, 30+3
add_result = np.add(arr1, arr2)

# Option 2: np.append (Concatenation)
# joins them into a longer list
append_result = np.append(arr1, arr2)

print(f"Array 1: {arr1}")
print(f"Array 2: {arr2}")
print(f"- " * 30)
print(f"Option 1 (np.add): {add_result}")
print(f"Option 2 (np.append): {append_result}")
```

```
Array 1: [10 20 30]
Array 2: [1 2 3]
-----
Option 1 (np.add):    [11 22 33]
Option 2 (np.append): [10 20 30  1  2  3]
```

Question 16

Option 4: np.subtract(arr1, arr2)

```
import numpy as np

arr1 = np.array([10, 20, 30, 40, 50])
arr2 = np.array([1, 2, 3, 4, 5])

# Option 4 (Corrected Spelling): Subtract arr2 from arr1
# 10-1, 20-2, 30-3, etc.
result = np.subtract(arr1, arr2)

print(f"Array 1: {arr1}")
print(f"Array 2: {arr2}")
print("-" * 30)
print(f"Result (arr1 - arr2): {result}")
```

```
Array 1: [10 20 30 40 50]
Array 2: [1 2 3 4 5]
-----
Result (arr1 - arr2): [ 9 18 27 36 45]
```

Question 17

Option 3: All the other 3 are rounding methods in NumPy

```
import numpy as np

# Let's test with a negative decimal to see the differences clearly
arr = np.array([-3.7, 3.7])

print(f"Original: {arr}")
print("-" * 30)

# Option 4: np.around (Standard rounding)
print(f"np.around(): {np.around(arr)}")
# -3.7 -> -4.0, 3.7 -> 4.0

# Option 2: np.trunc (Cut off decimal)
print(f"np.trunc(): {np.trunc(arr)}")
# -3.7 -> -3.0, 3.7 -> 3.0

# Option 1: np.fix (Round towards zero)
print(f"np.fix(): {np.fix(arr)}")
# -3.7 -> -3.0, 3.7 -> 3.0
```

```
Original: [-3.7  3.7]
-----
np.around(): [-4.  4.]
np.trunc():  [-3.  3.]
np.fix():    [-3.  3.]
```

Question 19

Option 4: [1 3 6]

```
import numpy as np

arr = np.array([1, 2, 3])

# Calculate Cumulative Sum
result = np.cumsum(arr)

print(f"Original Array: {arr}")
print(f"Cumulative Sum: {result}")
```

```
Original Array: [1 2 3]
Cumulative Sum: [1 3 6]
```